



Design First : OpenApi & AsyncApi

Et si vous arrêtiez de coder votre API avant de l'avoir pensée ?

Philippe Bousquet – Architecte & Leader Java Community France

Qui suis-je ?



Philippe Bousquet

- Architecte & Leader Java Community France
- Passionné de dev depuis mes 9 ans
- Près de 30 d'expérience professionnelle
- Pilote la Veille & l'Innovation Java

AQUINUM



[philippe bousquet](#)



[darken33](#)



DEVBOX™ France

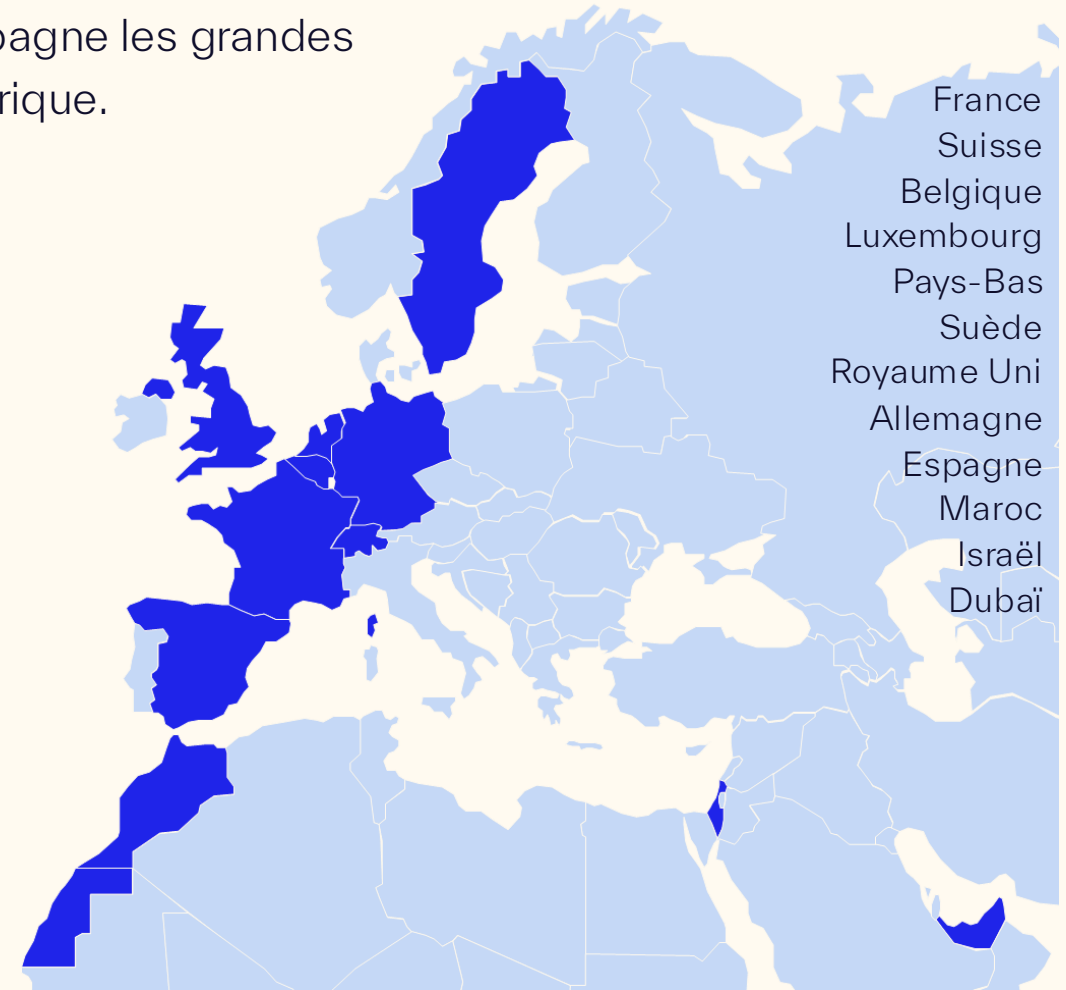
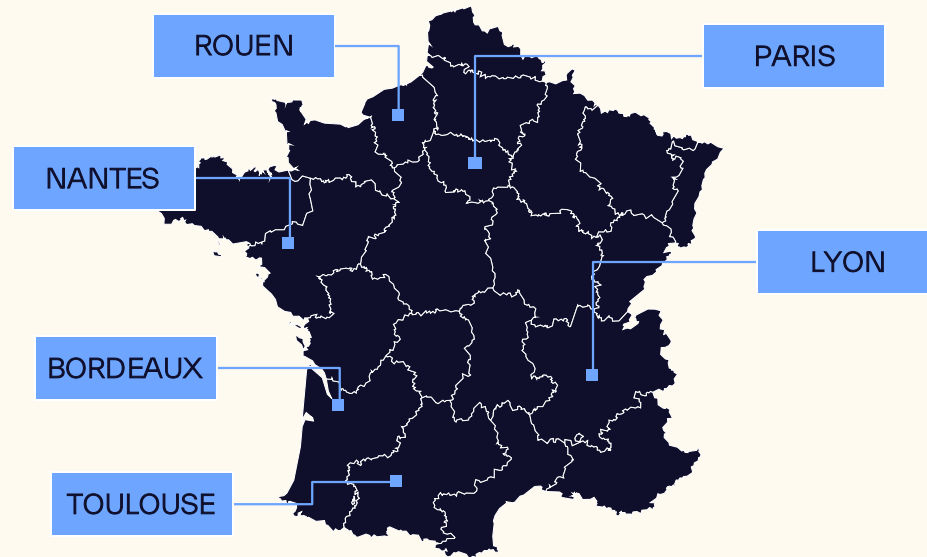


Le Groupe SQLI

AQUINUM



SQLI est un groupe européen de services numériques qui accompagne les grandes marques internationales dans la création de valeur grâce au numérique.



1990

Création du groupe

253

M€ en 2025

2200

Employés

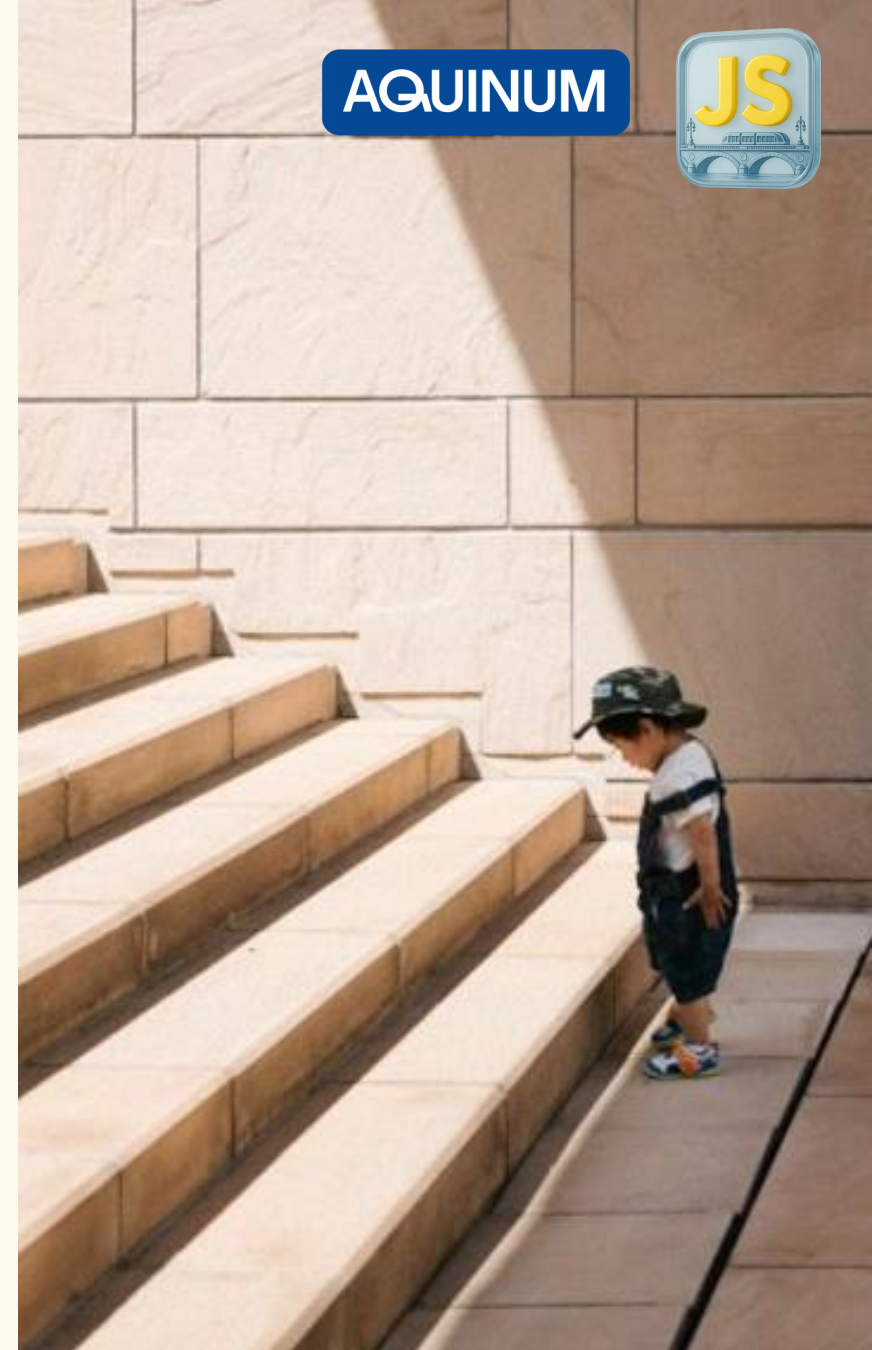
12

Pays



Une démarche API

Une démarche API



Une démarche API

Normalisation

- Définition des standards du SI et normalisation

Architecture

- Approche Domain Driven Design et Architecture Hexagonale

Spécification

- Approche Design First pour les spécifications et la documentation

Implémentation

- Se baser sur de bonnes pratiques de développement

Testabilité

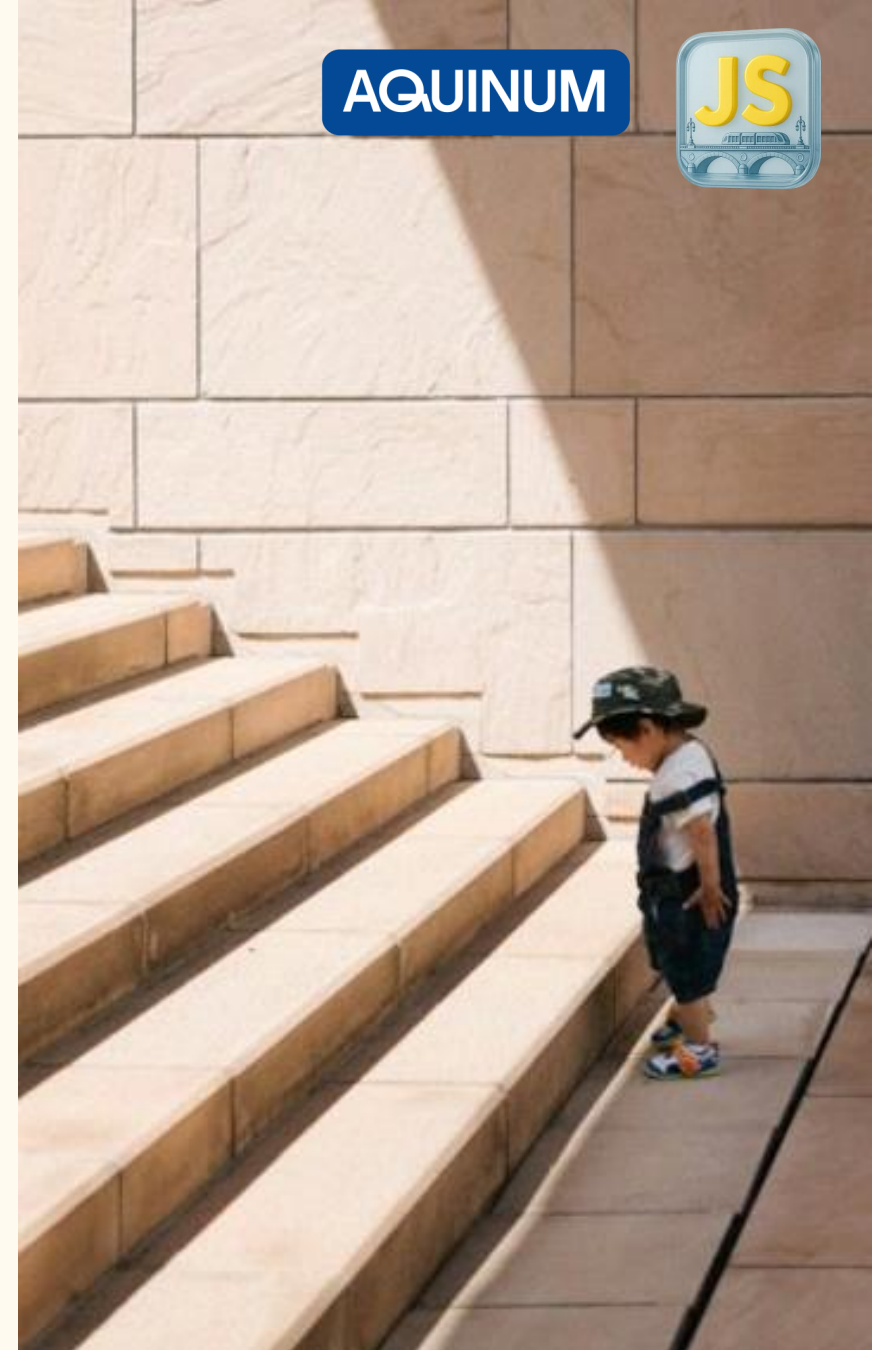
- Garantir la qualité au travers de la pyramide de tests

Management

- Mise en place d'outils de gouvernance des API

Observabilité

- Mise en place d'outils d'observabilité et de monitoring





Code First vs Design First

Code First

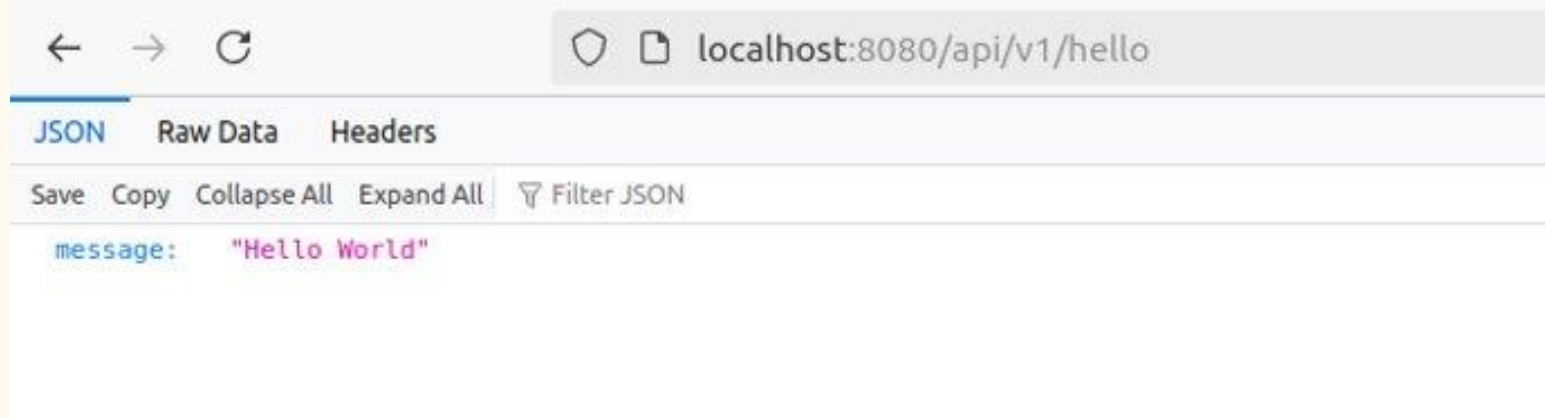
On ouvre son IDE et on code son API

```
@RestController
public class HelloApi {

    @RequestMapping(
        method = RequestMethod.GET,
        value = "/api/v1/hello",
        produces = { "application/json" }
    )
    public ResponseEntity<HelloDto> hello() {
        HelloDto result = new HelloDto();
        result.setMessage("Hello World");
        return ResponseEntity.ok(result);
    }
}
```


Code First

Et cela fonctionne



Aucune documentation d'API

Code First

On peut, il est vrai, ajouter des annotations OpenApi

```
@RequestMapping(
    ...
)
@Operation(
    operationId = "helloWithName",
    summary = "Saluer une personne en particulier",
    tags = { "Hello" },
    responses = {
        @ApiResponse(
            responseCode = "200",
            description = "OK",
            content = {
                @Content(
                    mediaType = "application/json",
                    schema = @Schema(implementation = HelloDto.class)
                )
            }
        ),
    },
)
public ResponseEntity<HelloDto> hello() {
    ...
}
```

AQUINUM



Design First

- Spécifier son API en amont
 - Implémenter plus efficacement son API
 - Faciliter l'intégration par les consommateurs
 - Simuler le fonctionnement de l'API
- Avoir une documentation en adéquation avec le code
 - AI Ready ?





OpenApi & AsyncApi

OpenApi & AsyncApi

Année			
2011	v1.0		
2014	v2.0		
2015	Smart Bear cède Swagger		
2017		v3.0.0	Fran Méndez
2019			v2.0.0
2021		v3.1.0	v2.2.0
2022			v2.5.0
2023			v3.0.0
2025		v3.2.0	
2026			

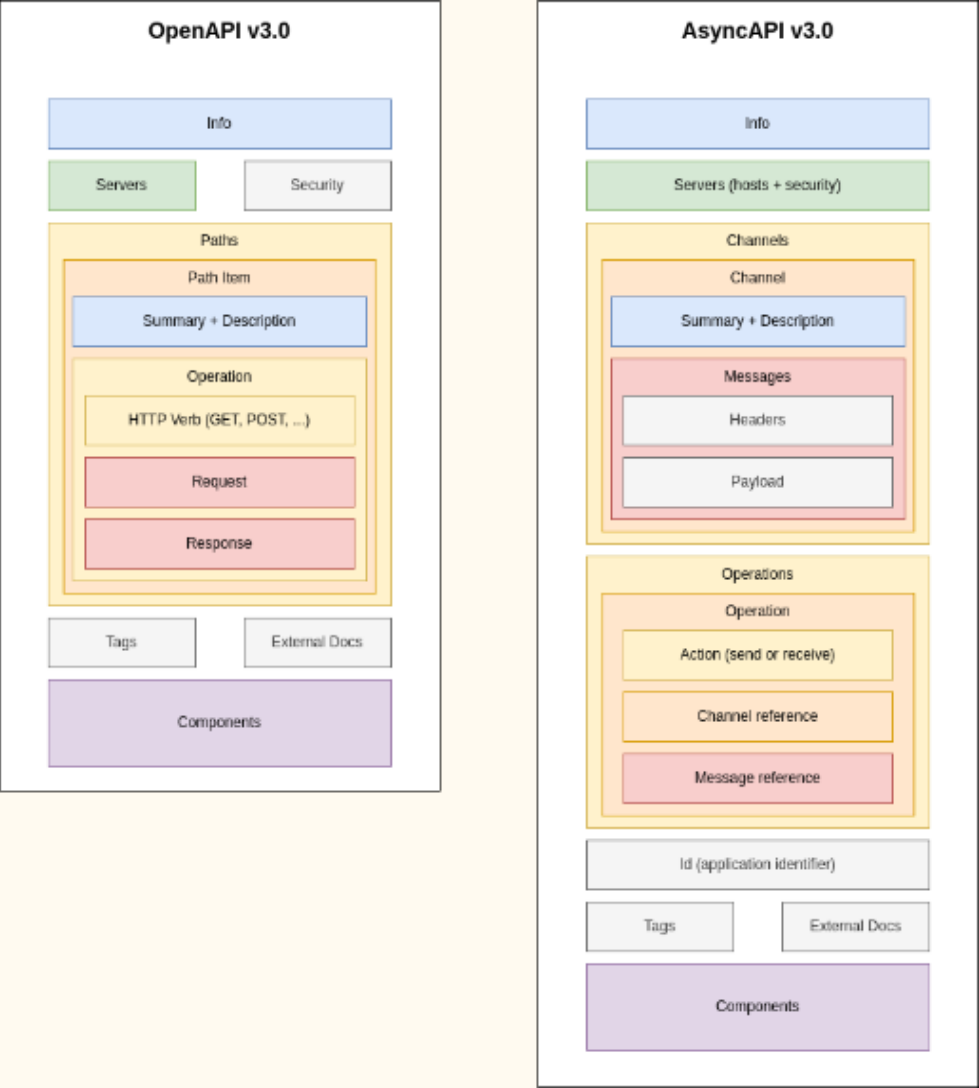
STANDARD

Les initiatives

		
Site web	https://www.openapis.org/	https://www.asyncapi.com/
Linux Foundation	✓	✓
Spécifications	✓	✓
Documentation	✓	✓
Outillage	✗	✓
Communauté	✓	✓



Les spécifications



Tooling

Tools for working with OpenAPI

There are many tools and resources available for working with OpenAPI. We've organized them into categories to help you find what you're looking for. If you're looking for a specific tool or resource, you can use the search bar at the top of the page.

If you feel like something is missing, check out our instructions on how to [contributing](#).

Annotations

Use annotations in your code to generate OpenAPI definitions, keeping the documentation close to the implementation.

Code generators

Tools to generate code from your OpenAPI Spec, or to generate an OpenAPI Spec from your code.

Converters

Various tools to convert to and from OpenAPI and other API description formats.

Data Validators

Check to see if API requests and responses are lining up with the API description.

Documentation

Render API Description as HTML (or maybe a PDF) so slightly less technical people can figure out how to work with the API

Domain-Specific Languages (DSLs)

Writing YAML by hand is no fun, and maybe you don't want a GUI, so use a Domain Specific Language to write OpenAPI in your language of choice.

Gateways

API Gateways and related tools that have integrated support for OpenAPI.

HTTP Clients

Tools and libraries for making HTTP requests to APIs usually with a fancy GUI experience.

IDEs and GUI Editors

Visual editors help you design APIs without

Learning

Whether generating documentation for a

<https://openapi.tools/>

AsyncAPI

AsyncAPI Tools Dashboard

Discover various AsyncAPI tools to optimize your journey! These tools are made by the community, for the community. Have an AsyncAPI tool you want to be featured on this list? Then follow the procedure given in the [Tool Documentation](#) file, and show up your AsyncAPI Tool card in the website.

Filter

Jump to Category

Search by name

Clear Filters

Code Generators

The following is a list of tools that generate code from an AsyncAPI document; not the other way around.

MultiAPI Generator

Open Source

This is a plugin designed to help developers automatizing the creation of code classes from YML files based on AsyncApi and OpenAPI. It is presented in 2 flavours Maven and Gradle

LANGUAGE

Java

TECHNOLOGIES

Groovy

Maven

View Github

<https://www.asyncapi.com/tools>

AQUINUM

JS

sqli

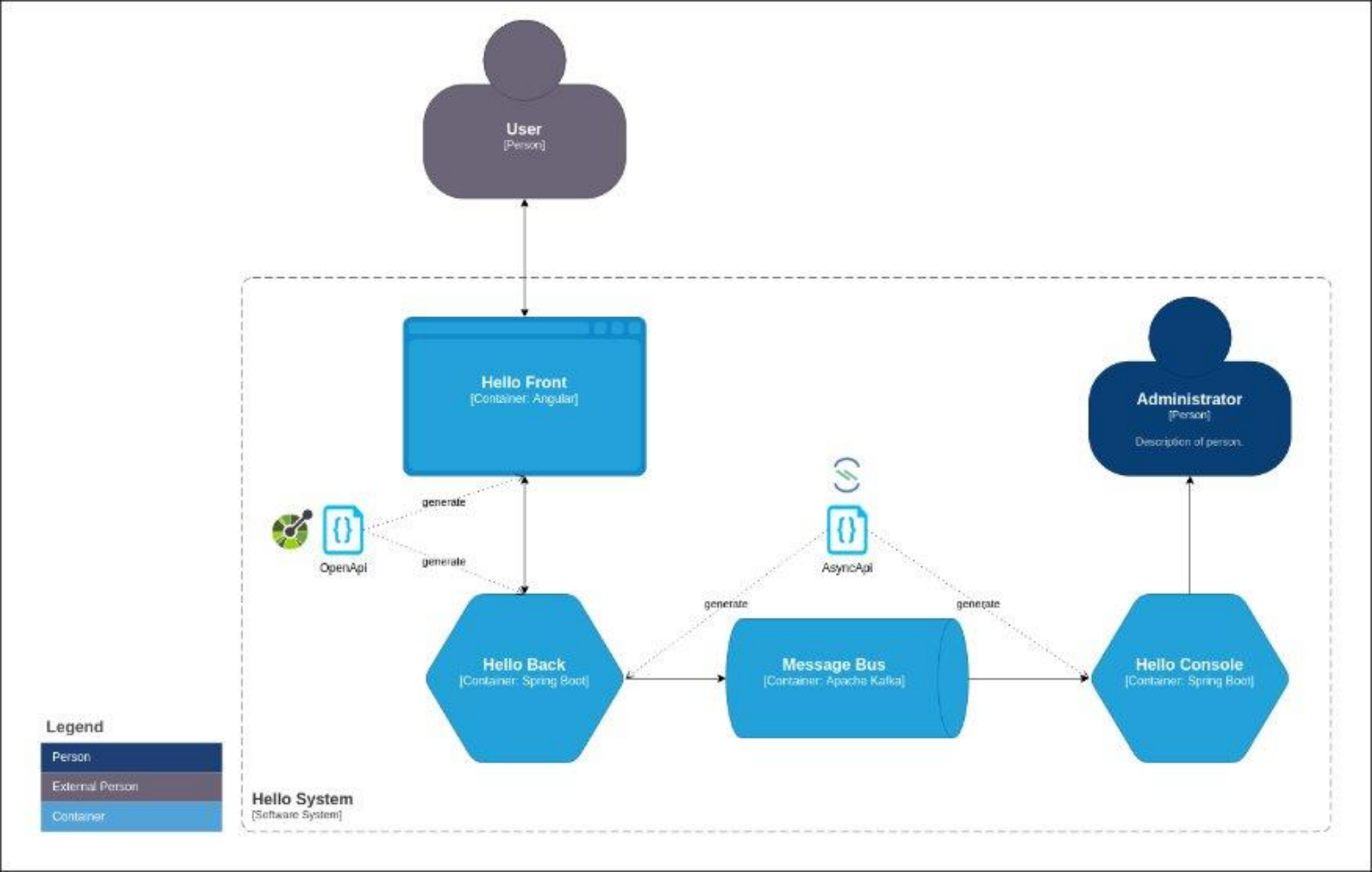
Document Name

03/05/2026	CO - Public	16
------------	-------------	----



Notre cas d'usage

Hello World



OpenApi : Designer

Swagger Editor

File Edit Generate Server Generate Client About

Sign in to Swagger Try Swagger

```
1 openapi: 3.0.1
2 info:
3   title: Hello API
4   description: Hello API
5   termsOfService: http://localhost:8080/swagger-ui.html
6   contact:
7     name: Philippe Bousquet
8     url: http://sqli.com
9     email: pbousquet@sqli.com
10  license:
11    name: Apache 2.0
12    url: http://www.apache.org/licenses/LICENSE-2.0
13  version: 1.0.0
14 #servers:
15 #- url: localhost:8080
16 tags:
17 - name: Hello
18   description: Hello API
19 paths:
20   /api/v1/hello:
21     get:
22       tags:
23       - Hello
24       description: Retourne un message de salutation générique.
25       summary: Saluer le monde
26       operationId: helloWorld
27       responses:
28         200:
29           description: OK
30           content:
31             application/json:
32               schema:
33                 $ref: '#/components/schemas/HelloDto'
34         "500":
35           description: Internal Server Error
36           content:
37             application/json:
38               schema:
39                 $ref: '#/components/schemas/ApiErrorResponse'
40       deprecated: false
```

Severity

Line

Code

Message

27

15000

Responses Object uses HTTP Status Codes outside of allowed IANA HTTP Status code registry

Hello API 1.0.0 OAS 3.0

Hello API

Terms of service

Philippe Bousquet - Website

Send email to Philippe Bousquet

Apache 2.0

Hello Hello API

GET /api/v1/hello Saluer le monde

GET /api/v1/hello/{name} Saluer une personne en particulier

Schemas

HelloDto

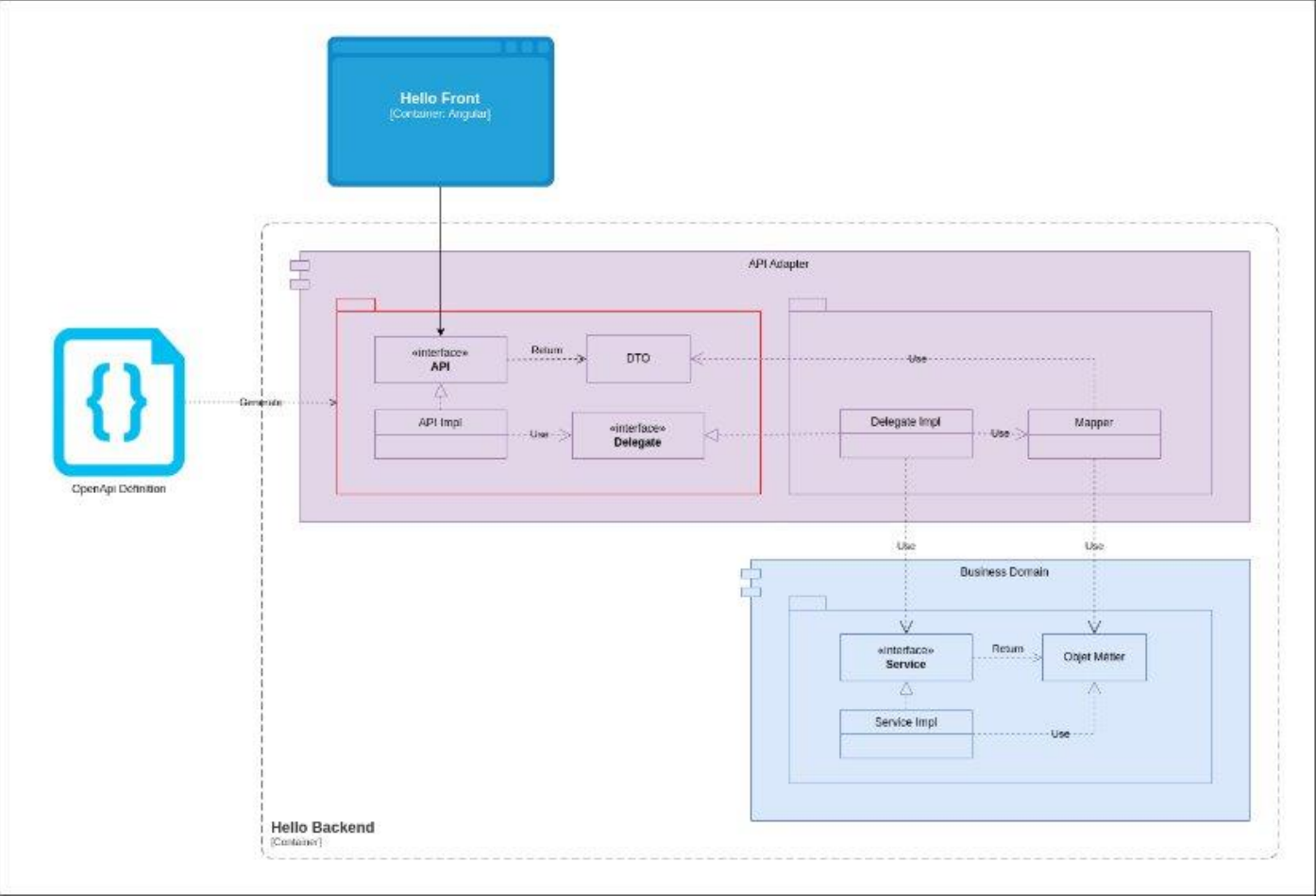
ApiErrorResponse

<https://editor.swagger.io/>



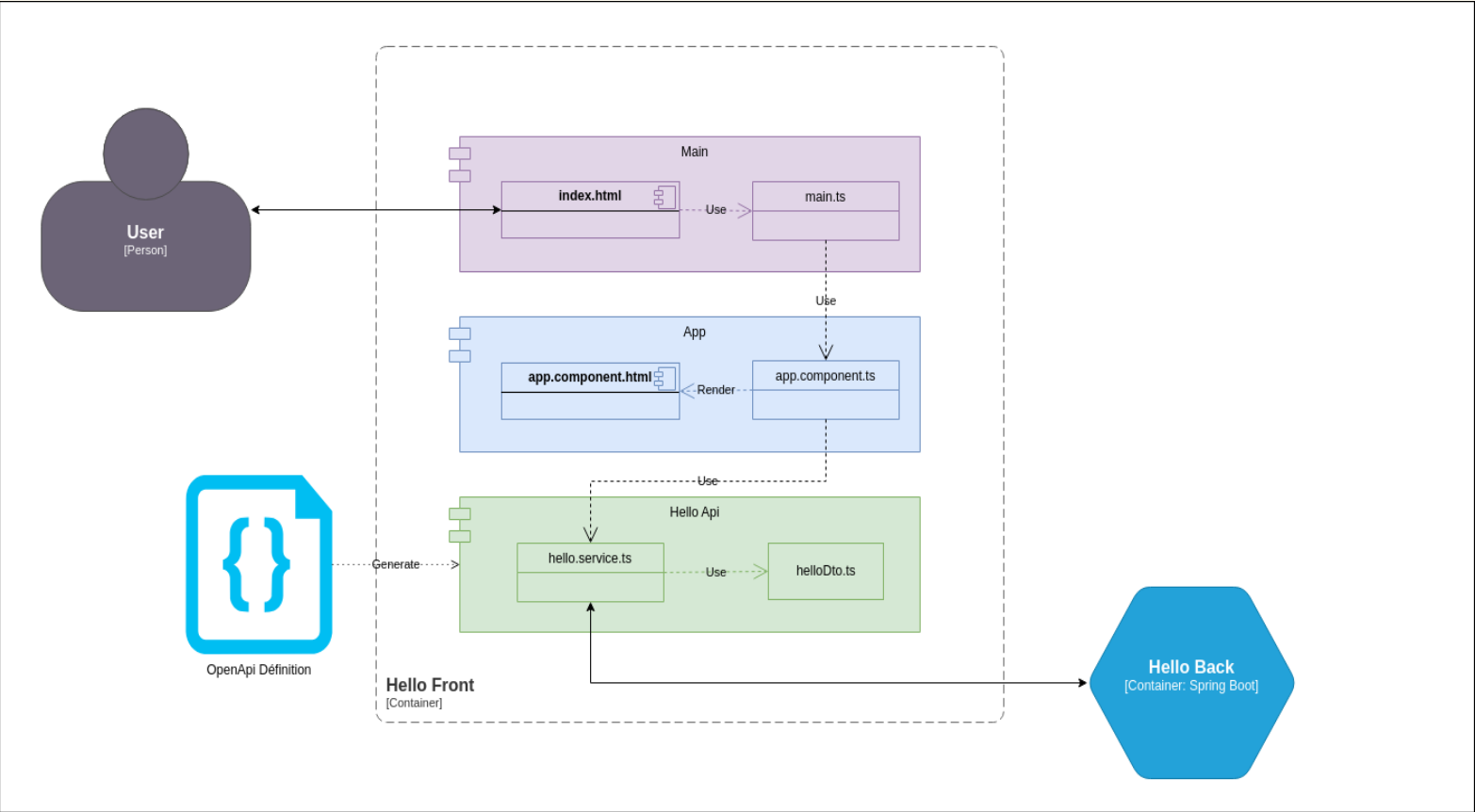
OpenApi : Génération Serveur

openapi-generator : DTOs, exposition API, pour plusieurs langages



OpenApi : Génération Client

openapi-generator : DTOs, client HTTP, pour plusieurs langages



AsyncApi : Designer

AQUINUM



The screenshot displays the AsyncApi Studio interface. On the left, a sidebar contains navigation tabs: Information, Servers, Channels, Operations, Messages, and Schemas. The main editor area shows a YAML definition for a Kafka API. The right panel provides a visual overview of the API, including its title, version, license, and a list of servers and operations.

```
1 asyncapi: 3.0.0
2 info:
3   title: Hello Kafka API
4   version: 1.0.0
5   description: |
6     Simple Hello Message API
7   license:
8     name: Apache 2.0
9     url: https://www.apache.org/licenses/LICENSE-2.0
10  defaultContentType: application/json
11  servers:
12    local:
13      host: localhost:9092
14      protocol: kafka
15      tags:
16        - name: local
17          description: 'This environment is meant for running internal
18  tests'
19  channels:
20    event.hello.v1:
21      address: event.hello.v1
22      messages:
23        helloMessage:
24          $ref: '#/components/messages/helloMessage'
25          description: The topic on which hello messages may be produced and
26            consumed.
27      operations:
28        receiveHelloMessage:
29          action: receive
30          channel:
31            $ref: '#/channels/event.hello.v1'
32          traits:
33            - $ref: '#/components/operationTraits/kafka'
34            - $ref: '#/channels/event.hello.v1/messages/helloMessage'
35        sendHelloMessage:
36          action: send
37          channel:
38            $ref: '#/channels/event.hello.v1'
```

Hello Kafka API 1.0.0

Simple Hello Message API

Servers

kafka://localhost:9092/ **KAFKA** **LOCAL**

Security:

SECURITY.PROTOCOL: **PLAINTEXT**

Operations

RECEIVE event.hello.v1

The topic on which hello messages may be produced and consumed.

Operation ID: **receiveHelloMessage**

Available only on servers:

local

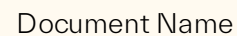
Operation specific information: **KAFKA** > Expand all: **Object**

Accepts the following message:

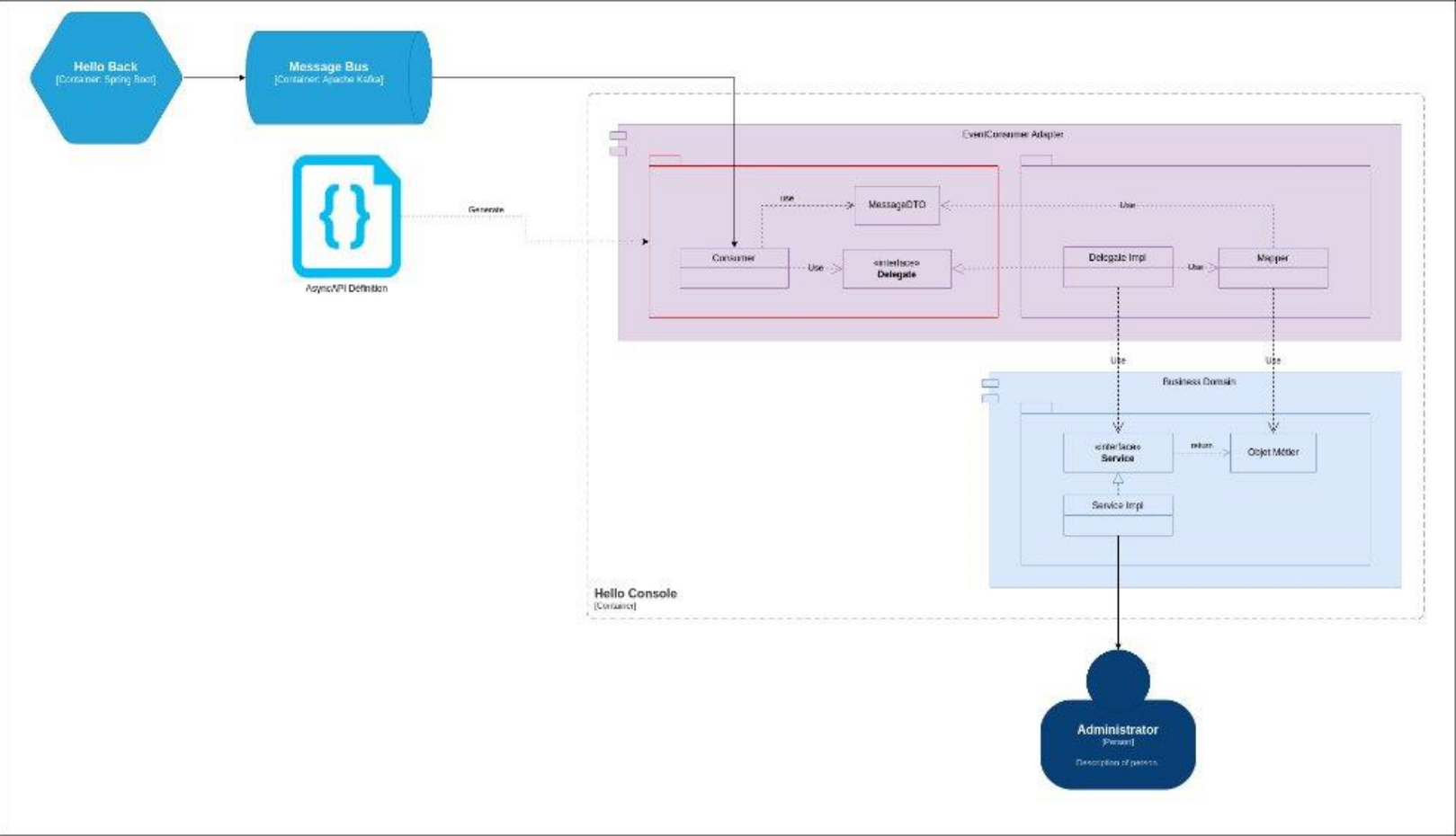
Hello Message: **helloMessage**

<https://studio.asyncapi.com/>

sqli



AsyncApi : Génération Consumer



La Démo

- Les Tools
- Le Design
- Les Mocks
- Le Front
- Le Back
- Le Receiver

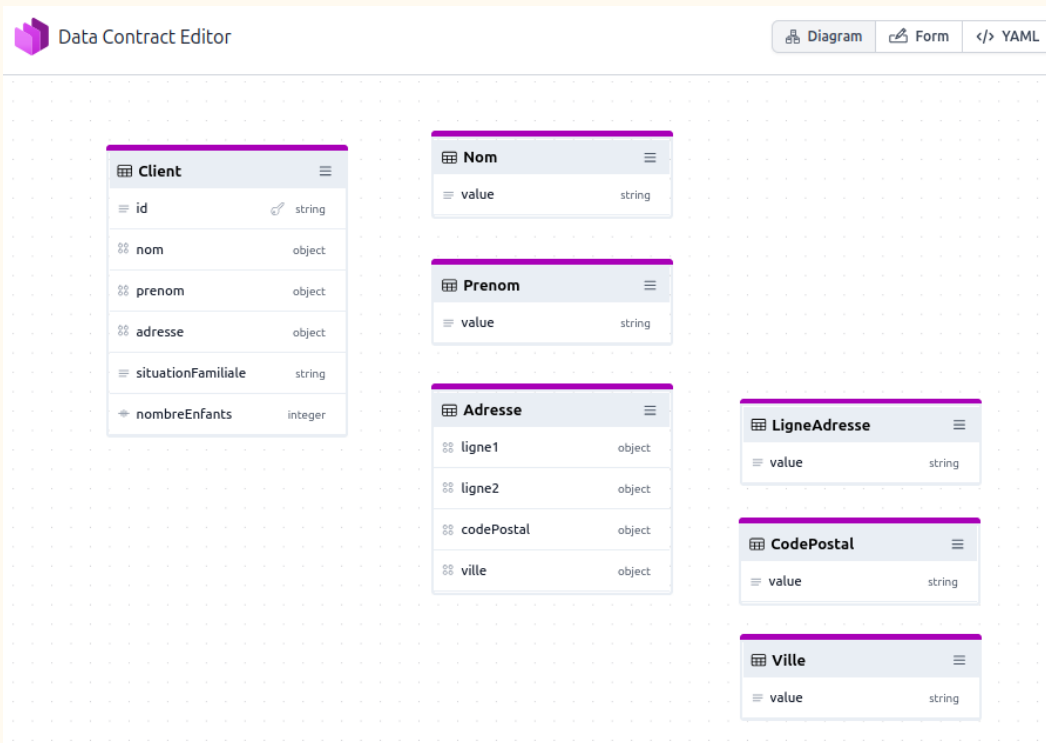




Aller plus loin

Open Data Contract Specification

Et si nous spécifions nos modèles de données (Base de Donnée, Domaine Métier, ...) ?

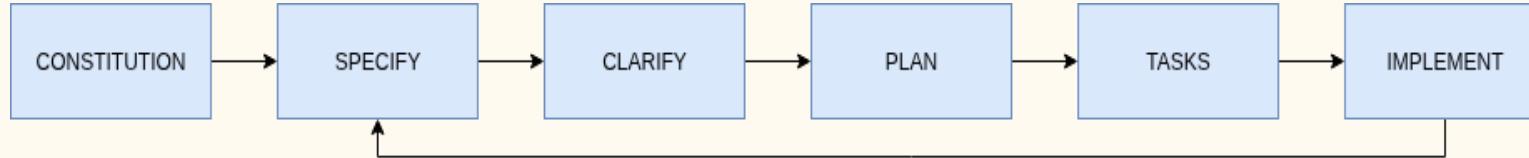


<https://bitol-io.github.io/open-data-contract-standard/v3.1.0/>

Github Copilot Spec-Kit

Specification Driven Development

Le Specification Driven Development canalise la créativité du vibe coding par des spécifications exécutables plutôt que par des prompts improvisés.



- Constitution : règles & architecture
- Spécification : intention fonctionnelle
- Clarification : éliminer les ambiguïtés
- Plan & tâches : délivrer de façon prédictible
- Implémentation : génération itérative

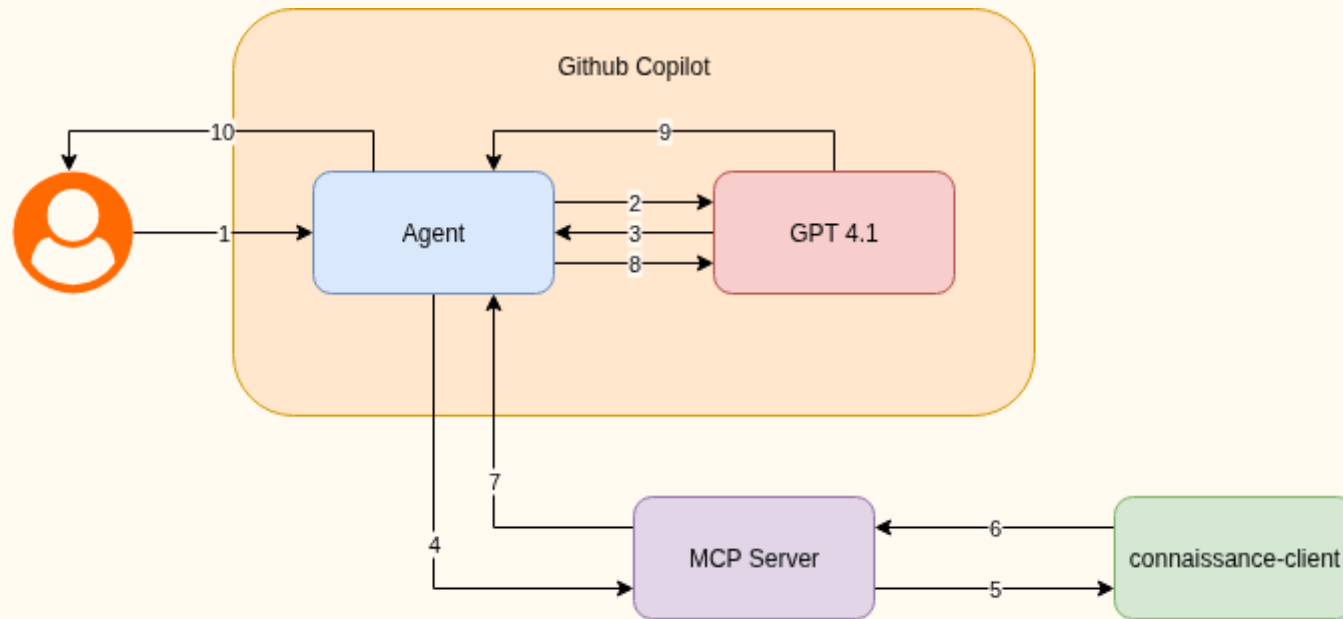
Pour mieux spécifier, **pensez design-first** : OpenAPI, AsyncAPI, Data Models avant le code.



MCP Servers

Faire interagir des LLM avec des APIs

Lors des ApiDays en décembre le message central était : **Les agents IA deviennent les premiers consommateurs d'APIs**



Une **API** bien documenté donnera du contexte au **LLM** (pensez OpenApi & AsyncApi).



sql

AQUINUM



Thank You



Elevate. Digitally